



LLM Interview Questions & Answers



Kalyan KS

100+ questions with detailed explanations

LLM Interview Questions and Answers

Author: Kalyan KS

Contents

Question 1: CNNs and RNNs don't use positional embeddings. Why do transformers use positional embeddings?	8
Question 2: Tell me the basic steps involved in running an inference query on an LLM.	10
Question 3: Explain how KV Cache accelerates LLM inference.	13
Question 4: How do you handle the large memory requirements of KV cache in LLM inference?	15
Question 5: Explain why subword tokenization is preferred over word-level tokenization in the Transformer model.	17
Question 6: Explain the trade-offs in using a large vocabulary in LLMs.	19
Question 7: After tokenization, how are tokens converted into embeddings in the Transformer model?	22
Question 8: Explain the role of token embeddings in the Transformer model.	24
Question 9: Explain the working of the embedding layer in the transformer model.	26
Question 10: What is the role of self-attention in the Transformer model, and why is it called "self-attention"?	28
Question 11: Explain how self-attention is computed in the Transformer model step by step.	30
Question 12: What is the purpose of the encoder in a transformer model?	33
Question 13: What is the purpose of the decoder in a transformer model?	34
Question 14: How does the encoder-decoder structure work at a high level in the Transformer model?	36
Question 15: What is the purpose of scaling in the self-attention mechanism in the Transformer model?	38
Question 16: Why does the Transformer model use multiple self-attention heads instead of a single self-attention head?	40
Question 17: How are the outputs of multiple heads combined and projected back in the multi-head attention in the Transformer model?	42
Question 18: How does masked self-attention differ from regular self-attention, and where is it used in a Transformer?	44
Question 19: What is the computational complexity of self-attention in the Transformer model?	46

Question 20: Discuss the pros and cons of the self-attention mechanism in the Transformer model.	48
Question 21: Explain how masking works in masked self-attention in Transformer.	50
Question 22: Explain why self-attention in the decoder is referred to as cross-attention. How does it differ from self-attention in the encoder?	52
Question 23: What is the softmax function, and where is it applied in the Transformer model?	55
Question 24: What is the purpose of residual (skip) connections in Transformer model layers?	57
Question 25: Why is layer normalization used, and where is it applied in Transformer?	59
Question 26: What is cross-entropy loss, and how is it applied during transformer training?	61
Question 27: Compare transformers and RNNs in terms of handling long-range dependencies.	64
Question 28: What are the fundamental limitations of the Transformer model?	66
Question 29: How do transformers address the limitations of traditional deep learning models like CNNs and RNNs?	68
Question 30: How do Transformers address the vanishing gradient problem?	70
Question 31: What is the purpose of the position-wise feed-forward sublayer in the Transformer model?	72
Question 32: Can you briefly explain the difference between LLM training and LLM inference?	74
Question 33: What is latency in LLM inference, and why is it important?	76
Question 34: What is batch inference, and how does it differ from single-query inference?	78
Question 35: How does batching generally help with LLM inference efficiency?	80
Question 36: Explain the trade-offs between batching and latency in LLM serving.	82
Question 37: How can techniques like mixture-of-experts (MoE) optimize inference efficiency?	84
Question 38: Explain the role of decoding strategy in LLM text generation.	86
Question 39: What are the different decoding strategies in LLMs?	87
Question 40: Explain the impact of the decoding strategy on LLM-generated output quality and latency.	90
Question 41: Explain the greedy search decoding strategy and its main drawback.	93
Question 42: How does Beam Search improve upon Greedy Search, and what is the role of the beam width parameter?	95
Question 43: When is a deterministic strategy (like Beam Search) preferable to a stochastic (sampling) strategy? Provide a specific use case.	98

Question 44: Discuss the primary trade-off between the computational cost and the output quality when comparing Greedy Search and Beam Search.	100
Question 45: When you set the temperature to 0.0, which decoding strategy are you using?	103
Question 46: How is Beam Search fundamentally different from a Breadth-First Search (BFS) or Depth-First Search (DFS)?	105
Question 47: Explain the criteria for choosing different decoding strategies.	108
Question 48: Compare deterministic and stochastic decoding methods in LLMs.	111
Question 49: What is the role of the context window during LLM inference?	113
Question 50: Explain the pros and cons of large and small context windows in LLM inference.	115
Question 51: What is the purpose of temperature in LLM inference, and how does it affect the output?	117
Question 52: What is autoregressive generation in the context of LLMs?	120
Question 53: Explain the strengths and limitations of autoregressive text generation in LLMs.	122
Question 54: Explain how diffusion language models (DLMs) differ from Large Language Models (LLMs).	124
Question 55: Do you prefer DLMs or LLMs for latency-sensitive applications?	126
Question 56: How does quantization affect inference speed and memory requirements	128
Question 57: Explain the concept of token streaming during inference.	130
Question 58: What is speculative decoding, and when would you use it?	132
Question 59: What are the challenges in performing distributed inference across multiple GPUs?	135
Question 60: How would you design a scalable LLM inference system for real-time applications?	137
Question 61: Explain the role of flash attention in reducing memory bottlenecks during inference.	140
Question 62: What is continuous batching, and how does it differ from static batching?	142
Question 63: What is mixed precision (e.g., FP16) and why is it used during inference?	143
Question 64: Differentiate between online and offline LLM inference deployment scenarios and discuss their respective requirements.	145
Question 65: Explain the throughput vs. latency tradeoff in LLM inference.	147
Question 66: What are the various bottlenecks in a typical LLM inference pipeline when running on a modern GPU?	149
Question 67: How do you measure LLM inference performance?	151
Question 68: What are the different LLM inference engines available? Which one do you prefer?	153

Question 69: What are the challenges in LLM inference?	156
Question 70: What are the possible options for accelerating LLM inference?	158
Question 72: Explain the reason behind the effectiveness of Chain-of-Thought (CoT) prompting.	162
Question 73: Explain the trade-offs in using CoT prompting.	164
Question 74: What is prompt engineering, and why is it important for LLMs?	166
Question 75: What is the difference between zero-shot and few-shot prompting?	168
Question 76: What are the different approaches for choosing examples for few-shot prompting?	170
Question 77: Why is context length important when designing prompts for LLMs?	172
Question 78: What is a system prompt, and how does it differ from a user prompt?	174
Question 79: What is In-Context Learning (ICL), and how is few-shot prompting related?	176
Question 80: What is self-consistency prompting, and how does it improve reasoning?	178
Question 81: Why is context important in designing prompts?	180
Question 82: Describe a strategy for reducing hallucinations via prompt design.	182
Question 83: How would you structure a prompt to ensure the LLM output is in a specific format, like JSON?	184
Question 84: Explain the purpose of ReAct prompting in AI Agents.	186
Question 85: What are the different phases in LLM development?	188
Question 86: What are the different types of LLM fine-tuning?	190
Question 87: What role does instruction tuning play in improving an LLM's usability?	192
Question 88: What role does alignment tuning play in improving an LLM's usability?	194
Question 89: How do you prevent overfitting during fine-tuning?	196
Question 90: What is catastrophic forgetting, and why is it a concern in fine-tuning?	198
Question 91: What are the strengths and limitations of full fine-tuning?	200
Question 92: Explain how parameter efficient fine-tuning addresses the limitations of full fine-tuning.	202
Question 93: When might prompt engineering be preferred over task-specific fine-tuning?	204
Question 94: When should you use fine-tuning vs. RAG?	206
Question 95: Explain the limitations of using RAG over LLM fine-tuning.	208
Question 96: Explain the limitations of using LLM fine-tuning over RAG.	210
Question 97: When should you prefer task-specific fine-tuning over prompt engineering?	212

Question 98: What is LoRA, and how does it work at a high level?	215
Question 99: Explain the key ingredient behind the effectiveness of the LoRA technique.	217
Question 100: What is QLoRA, and how does it differ from LoRA?	219
Question 101: When would you use QLoRA instead of standard LoRA?	221
Question 102: How would you handle LLM fine-tuning on consumer hardware with limited GPU memory?	223
Question 103: Explain different preference alignment methods and their trade-offs.	225
Question 104: What is gradient accumulation, and how does it help with fine-tuning large models?	228
Question 105: What are the possible options to speed up LLM fine-tuning?	230
Question 107: What is the difference between casual language modeling and masked language modeling?	233
Question 108: How do LLMs handle out-of-vocabulary (OOV) words?	235
Question 109: In the context of LLM pretraining, what is Scaling Law?	237
Question 110: Explain the concept of Mixture-of-Experts (MoE) architecture and its role in LLM pretraining.	239
Question 111: What is model parallelism, and how is it used in LLM pre-training?	241
Question 112: What is the significance of self-supervised learning in LLM pretraining?	244

LLM Interview Questions and Answers

Copyright © 2026 Kalyan KS

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

First Edition

Published: January 2026

Author: Kalyan KS, NLP Researcher and Consultant. ([LinkedIn](#) and [Twitter](#))

Disclaimer

The information in this book is distributed on an “As Is” basis, without warranty. While every precaution has been taken in the preparation of this work, the author shall not have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained in this book.

Question 1: CNNs and RNNs don't use positional embeddings. Why do transformers use positional embeddings?

Answer

CNNs and RNNs don't use positional embeddings because these models inherently capture positional information through convolutional filters or recurrence over time steps. However, the Transformer model uses the self-attention mechanism, which processes all tokens in parallel without any notion of sequence or order.

Positional embeddings inject explicit information about each token's position in the sequence, allowing transformers to understand word order and relative distances between tokens. This helps them model sequential dependencies and sentence structure effectively, compensating for the parallel and order-agnostic nature of self-attention mechanisms.

Detailed Explanation

Take the sentence:

“Cats chase mice.”

If we shuffle the words:

“Mice chase cats.”

The meaning completely changes. This simple example shows why any language model must understand **word order**, not just the words themselves.

Why RNNs Don't Need Positional Embeddings

An RNN (Recurrent Neural Network) processes a sentence sequentially, reading one word at a time from left to right. For example, given the sentence “I love apples,” the model first reads “I,” then “love,” and finally “apples.”

- Step 1 → “I”
- Step 2 → “love”
- Step 3 → “apples”

Because each word is processed after the previous one, the model inherently knows that “I” came before “love,” and “love” came before “apples.” This sequential processing gives RNNs a natural understanding of order, which is why they do not require positional embeddings.

Why CNNs Don't Need Positional Embeddings

A CNN (Convolutional Neural Network) processes text using sliding windows that move across the sentence. Consider the sentence “The dog barked loudly.” A convolutional window might first capture “The dog barked” and then slide to capture “dog barked loudly.”

- Window 1 → “The dog barked”
- Window 2 → “dog barked loudly”

These windows preserve local word order—“dog” appears before “barked,” and “barked” appears before “loudly.” By learning patterns within these overlapping windows, CNNs implicitly learn positional relationships, making explicit positional embeddings unnecessary.

Why Transformers Are Different

Transformers operate very differently. They process all words in a sentence at the same time, rather than sequentially or through sliding windows. For the sentence “She ate cake,” a Transformer sees the tokens “She,” “ate,” and “cake” in parallel, without any inherent order. To the self-attention mechanism, this can look like an unordered set of words. If the words are shuffled, the model would not automatically know that the meaning has changed.

Why Transformers Need Positional Embeddings

Because self-attention is order-agnostic, a Transformer has no built-in sense of which word came first, which came next, or how far apart words are. Positional embeddings solve this problem by adding position information to each word's representation.

Word	Position	Positional embedding meaning
She	1	“I'm the first word.”
ate	2	“I'm after She.”
cake	3	“I'm the third word.”

These positional embeddings are added to the token embeddings:

$$\text{Token Embedding (Meaning)} + \text{Positional Embedding (Position)} = \text{Final embedding}$$

With this additional information, the Transformer can correctly interpret that “She” comes before “ate,” and “ate” comes before “cake.” As a result, it can distinguish between “She ate cake” and “Cake ate she.” .

Question 2: Tell me the basic steps involved in running an inference query on an LLM.

Answer

Running an inference query on an LLM involves the following steps:

- **Tokenization:** The input text is first broken down into tokens, and then the tokens are mapped to their IDs in the model's vocabulary.
- **Prefill Phase:** The embedding layer converts these token IDs into embeddings, and then the decoder layers process these embeddings simultaneously to compute intermediate states (keys and values). This parallel processing establishes the context for generating new tokens.
- **Decoding Phase:** The model generates output tokens one at a time autoregressively, each based on previously generated tokens and cached states, continuing until a stopping condition is met.
- **Detokenization:** Finally, the generated tokens are converted back into human-readable text.

Detailed Explanation

Tokenization — Breaking Text into Small Pieces

Before a large language model (LLM) can understand any text, the text must first be broken down into smaller units called *tokens*. A token can be a full word, part of a word, or even a punctuation mark. For example, consider the input text:

“Hello world!”

A tokenizer might split this text into the following tokens:

- “Hello”
- “ world”
- “!”

Each of these tokens is then mapped to a numerical ID, since all internal processing is numerical.

Example token IDs:

- “Hello” $\rightarrow$ 1564

- “ world” $\rightarrow$ 982
- “!” $\rightarrow$ 299

After tokenization, the original text is represented numerically as:

[1564, 982, 299]

This numerical sequence is what is actually fed into the model.

Prefill Phase — The Model Reads the Entire Input at Once

Once tokenization is complete, the model enters what is known as the *prefill phase*. In this phase, the LLM processes the entire input sequence at the same time, effectively “reading” the full sentence in one pass. This phase involves the following steps:

- Each token ID is converted into an embedding, which is a vector of numbers representing semantic meaning.
- All input tokens are processed in parallel, rather than sequentially.
- The model constructs a memory-like structure using keys and values, commonly referred to as the **KV cache**.

These keys and values represent what the model has understood from the input so far and will be reused during generation to avoid recomputing past information.

Decoding Phase — The Model Generates One Token at a Time

After the input has been fully understood, the model moves to the *decoding phase*, which is where the actual response is generated. Unlike the prefill phase, decoding happens sequentially.

Consider the question:

“What is 2 + 2?”

The model might generate the following tokens, one after another:

- “The”
- “ answer”
- “ is”
- “ 4”

Detokenization — Converting Tokens Back Into Text

Once the model has finished generating tokens, the final step is *detokenization*. This step converts numerical token IDs back into human-readable text.

For example:

Generated token IDs: [1011, 938, 117, 21]

Detokenized output: **“The answer is 4”**

This detokenized text is what the user ultimately sees as the model’s response.